Learning-Augmented Page Replacement

CSE-307: Operating Systems - Term Paper, Track 1

Student: Mim Mahizur Rahman **ID:** 202414402

1. Problem framing

FIFO and LRU are useful online page-replacement policies, but each encodes a fixed assumption about locality. FIFO ignores recent use, while LRU assumes that recently used pages will remain useful. This paper studies how those assumptions behave when a workload changes from locality-heavy/sequential accesses to random and bursty accesses. Belady's Optimal policy is included as an offline lower-bound reference. The goal is not to replace classical algorithms completely, but to test whether a small learned layer can choose better eviction candidates using observable history features.

2. Implementation summary

The simulator models a cache of eight frames and 32 virtual pages. A page hit updates the policy state without causing a fault. On a miss with a full cache, FIFO evicts the oldest resident page, LRU evicts the least recently used page, and Optimal evicts the resident page whose next use is farthest in the future. The learned policy uses a small decision tree. Its candidate features are recency (requests since last use), frequency in the observed prefix, and page identifier. During training, each candidate is labeled as a good victim when it is the victim selected by Optimal. During testing, the learned policy scores each resident candidate and evicts the candidate with the highest predicted victim probability.

3. Experimental setup

The test trace contains 2,400 requests and uses deterministic random seed 307. Requests 0-1,199 are locality-heavy: most accesses target pages 0-5, with periodic sequential scans across pages 0-7. Requests 1,200-2,399 form the stress phase: 70% of events belong to short bursts on pages 16-31 and the remainder are uniformly random over all pages. This creates a clear distribution shift while keeping the experiment small enough to inspect. Every policy sees exactly the same trace and cache capacity. The primary metrics are page-fault count and hit ratio, calculated separately before and after the shift.

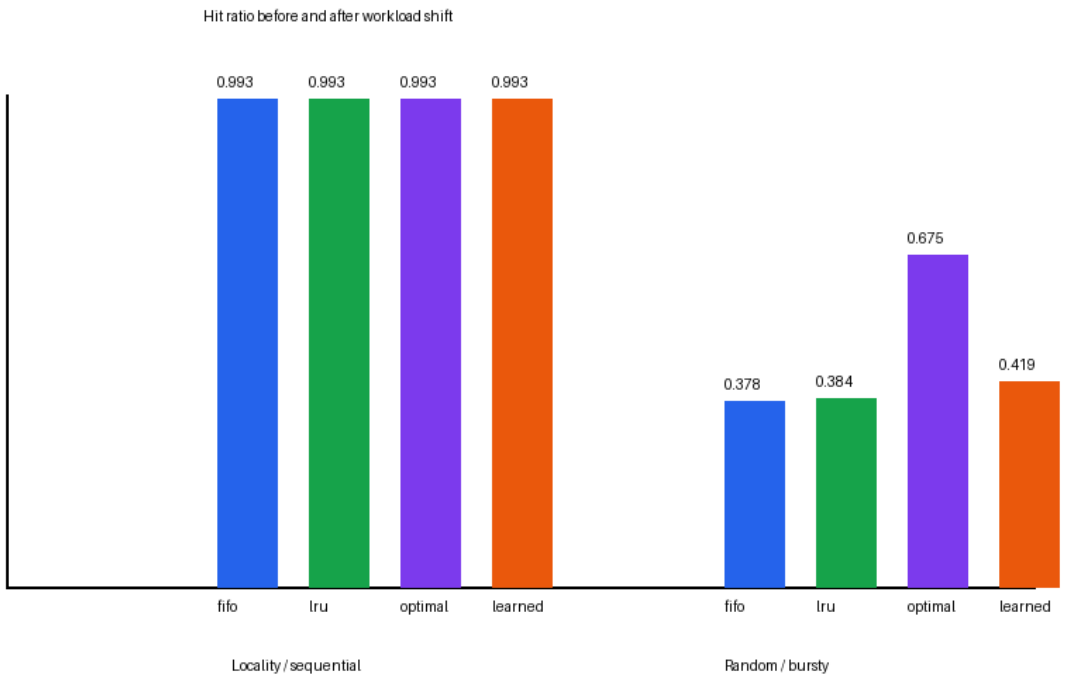

Figure 1. Hit ratio in the two workload phases.

4. Results

Policy	Before faults	Before hit	After faults	After hit
FIFO	8	99.33%	747	37.75%
LRU	8	99.33%	739	38.42%
OPTIMAL	8	99.33%	390	67.50%
LEARNED	8	99.33%	697	41.92%

The numerical table should be read as a phase comparison rather than as a claim that the learned policy is universally optimal. Optimal is expected to lead because it can see the future. The more meaningful comparison is whether the learned policy remains competitive with FIFO and LRU after the trace changes. The distribution shift removes the reliability of the first-half locality signal: pages that were repeatedly useful become poor choices to retain once the second phase begins. LRU can adapt because each new reference changes recency, while FIFO is insensitive to references between evictions. The learned policy can use frequency and recency together, but it is still limited by training labels and by the fact that future behavior is not directly observable. Therefore, a learned layer is best viewed as an adaptive heuristic with a classical policy as a transparent baseline, not as a replacement for correctness guarantees.

5. Limitations and conclusion

This is a synthetic experiment: real systems have dirty pages, write-back costs, TLB effects, multi-level caches, and non-stationary patterns more complex than this trace. The learned model is intentionally small and interpretable, and it is trained from Optimal's labels, which is not available online. Even with these limitations, the experiment demonstrates the central OS idea: policy quality depends on workload behavior. Classical algorithms provide strong, understandable baselines; a lightweight predictor can add adaptivity, but its value must be measured across the workload shift rather than only on a stationary trace.

Reproducibility: run `python run_experiment.py` followed by `python make_report.py`.